DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

Gradle's daemon learns to check ID before opening the parcel, and JEP 544 lands in JDK 28

Issue #10 | Wed, 14 Oct 2026 | ⏱️ ~36 min read | Level 3/5

☕ **Today's two stories**

1. **Gradle 9.8.1 and 8.14.6 fix three high-rated security flaws, and you should update your wrapper.** Two of them come from the same classic Java mistake: the Gradle daemon deserialized incoming messages before it checked who sent them. The third let a build quietly skip a repository it couldn't securely connect to and fetch the dependency from the next one instead. Here's how the daemon talks to your IDE and test workers, why Java deserialization is so dangerous, what changed byte by byte, and who's actually at risk.
2. **JEP 544, ahead-of-time code compilation, is now in the JDK 28 codebase.** In issue #2 it was only *targeted*. On 8 October the 8,600-line implementation landed. We read the code: when the native code is actually produced, how the JVM eventually replaces it with fresh JIT code, and the long list of things that make it quietly switch itself off.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 Gradle's daemon learns to check ID before opening the parcel

BUILD TOOLS · GRADLE 9.8.1 (7 OCT 2026) AND 8.14.6 (8 OCT 2026) · 3 HIGH-RATED ADVISORIES, NO CVE IDS · ~15 MIN READ

Run `./gradlew build` once and close the terminal. Something keeps running. A Java process you never explicitly started sits on your machine for up to three hours, listening on a local network port, waiting for the next build. That process is the reason Gradle's second build is so much faster than the first. Last week, Gradle admitted it had a problem: anyone who could reach that port could hand it a parcel, and it would open the parcel *before* checking who'd sent it. In Java, opening the parcel can be enough to run someone else's code.

WHAT HAPPENED

On 7 October 2026, Gradle released **9.8.1**, the first patch release for Gradle 9.8.0. The release notes say it "addresses three high rated vulnerabilities" and recommend that everyone upgrade.[1] A day later, **Gradle 8.14.6**, the sixth patch release of the 8.14 line, shipped the same three fixes for teams still on Gradle 8.[2,5] The three advisories are:

1. **"Unauthenticated worker-to-daemon channel deserializes untrusted Java objects"** (GHSA-mvvg-497x-hmj8).[1]

2. **"Deserialization of untrusted Java objects before authentication in the client - daemon communication"** (GHSA-xwqc-3h47-hg64).[1,3]
3. **"Failure to disable repositories failing to establish an SSL connection can expose builds to malicious artifacts"** (GHSA-j5m7-59rp-24f5).[1]

The client-daemon advisory, the one we could open in full, is rated High with a **CVSS** ≈ *Common Vulnerability Scoring System: the standard 0–10 severity score for security flaws. Version 4 is the newest* v4 score of 8.6. It affects Gradle versions below 8.14.6 and 9.0.0 through 9.8.0, and is fixed in 8.14.6 and 9.8.1. Gradle also offers backports to 9.7.2, 9.6.2, 9.5.2, 9.4.2, 9.3.2, 9.2.2, 9.1.2 and 7.6.7, but only through its paid Gradle Security Subscription.[3] According to an editorial review of the advisories by The Machine Herald, the worker advisory scores 7.3 and the repository advisory 8.6, and none of the three had a CVE number.[4] The client-daemon advisory itself lists "No known CVE".[3]

The patch also fixes three regressions from 9.8.0, including one where Gradle removed `java.util.logging` handlers installed by a custom `LogManager`, which broke Quarkus test log capture.[1]

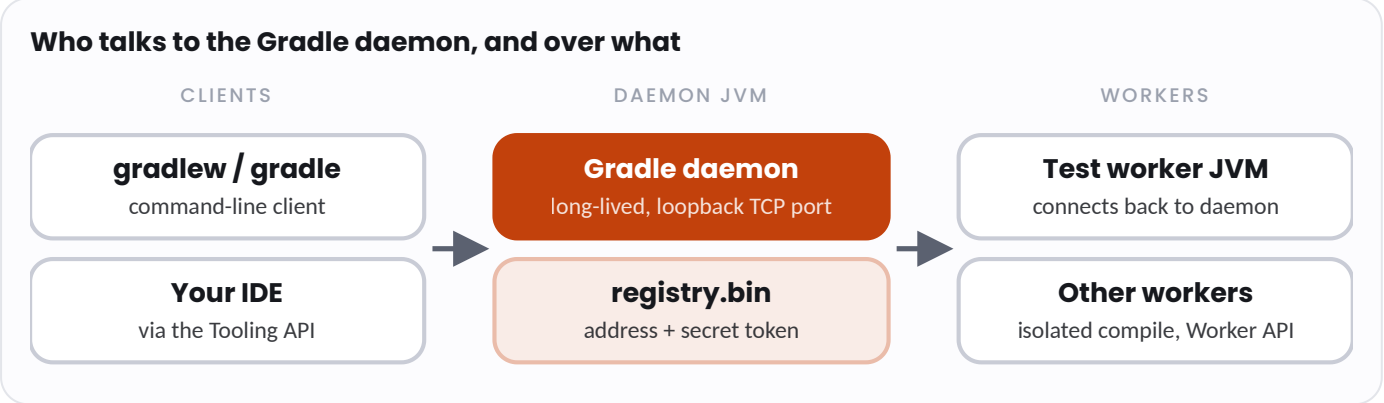
THE BACKGROUND YOU NEED

The daemon, the client and the workers

When you type `gradle` or `./gradlew`, the process you start is only a thin **client**. The real work happens in the **Gradle daemon** ≈ *a long-lived background JVM that runs your builds. It stays alive between builds so that loaded classes, caches and JIT-compiled code can be reused*. Gradle has enabled it by default since Gradle 3.0. The client looks for a compatible idle daemon and starts one only if none exists, and an idle daemon stops itself after three hours.[10] Your IDE is a client too: IntelliJ IDEA and other tools drive builds through the Gradle **Tooling API** ≈ *the library IDEs and other programs use to run Gradle builds and query project models programmatically*.[3]

The daemon often starts further JVMs of its own, called **worker processes** ≈ *separate JVMs the daemon starts for parts of a build, such as running your tests or compiling in an isolated process. They connect back to the daemon to receive work and send results*. Your test tasks run in them, for example.

All of these processes talk to each other over **loopback** ≈ *the network interface (127.0.0.1) that only reaches the same machine. Traffic never leaves the computer* TCP connections.[3] The daemon records its port in a registry file under your Gradle user home (`~/.gradle/daemon/<version>/registry.bin`), so clients can find it.[10]



A loopback port can't be reached from the internet. But it *can* be reached by any other program, or any other user, on the same machine. Gradle knew this and had already protected the client-daemon channel with a secret. When a daemon starts, it generates a random 16-byte **authentication token** ≈ *a secret value a client must present to prove it's allowed to use a service* with `SecureRandom`, with a comment saying the client must provide it "in any requests it makes".[9] The question was *when* the daemon checked it.

Why "deserialize first, check later" is dangerous in Java

Java serialization ≈ *the JDK's built-in mechanism (ObjectOutputStream/ObjectInputStream) for turning an object graph into bytes and back again* looks harmless. You write objects into a stream, and the other side reads them back. The catch is that **the bytes decide which classes get instantiated**, not the code that reads them. Oracle's own guidance puts it bluntly: "Deserialization is code execution because the readObject method of the class that is being deserialized can contain custom code."[13]

Any `Serializable` class can define a private `readObject` method, and `ObjectInputStream` calls it while it rebuilds the object, before your code gets the result back. So a type check after `readObject()` returns is too late. We compiled and ran this on JDK 21 to show the order of events:

```
// JDK 21, plain Java. Gadget has a custom readObject().
Object msg = in.readObject();         // Gadget.readObject() runs HERE
if (!(msg instanceof Command c) || !c.token().equals("s3cr3t"))
    throw new SecurityException("bad token");   // too late
```

Run against a serialized `Gadget`, it prints `Gadget.readObject() ran!` first, and only then `rejected: bad token`. The rejection works, but the attacker's chosen class has already executed code inside your JVM.

A real attack doesn't need its own class on your classpath. It strings together existing classes from popular libraries whose `readObject` methods, combined, end up calling something powerful, such as a method found by reflection. These sequences are called **gadget chains** ≈ *chains of existing classes on the classpath whose deserialization logic can be combined to perform an action the attacker chooses, such as running a command*. The idea went mainstream in 2015. At AppSecCali, Gabriel Lawrence and Chris Frohoff presented "Marshalling Pickles", and Frohoff's tool **ysoserial** generated ready-made payloads for gadget chains in libraries such as Apache Commons Collections, Spring and the Groovy runtime.[14,15] Foxglove Security then used it against WebSphere, JBoss, Jenkins, WebLogic and OpenNMS. Apache Commons responded by disabling deserialization of `InvokerTransformer`, a class that calls methods by reflection, but warned that a hardened library alone "will not" make an application safe.[14] The ysoserial README makes the same point: the vulnerability is the unsafe deserialization, not the presence of gadgets.[15]

So for a decade the rule has been: **never deserialize bytes from someone you haven't authenticated.**

Repositories, and what "disabling" one means

The third advisory is about dependency resolution. Your build lists repositories in order, perhaps your company's internal mirror first and Maven Central second. For each dependency, Gradle asks them in turn. If the first answers "not found", Gradle moves on to the next. That's normal.

But if a repository is *broken*, Gradle does something stricter. Its documentation says that after repeated retries or "specific fatal errors" the repository is **disabled** for the rest of the build, and Gradle "will not continue to the next repository in the list unless told otherwise". That usually fails the build, deliberately, because "subsequent builds could produce different results once the repository is back online".[11] Silently getting a dependency from a different source than usual isn't just a reproducibility problem. It's how **dependency confusion** ≈ *an attack where a malicious package with the same name as a private one is published to a public repository, so a build that falls through to the public repository downloads the attacker's version* works.

HOW IT WORKS

Fix 1: the client-daemon handshake now carries the token

Before 9.8.1, a new connection worked like this. The client opened a TCP connection and sent a fixed **preamble** ≈ *a fixed sequence of bytes sent at the start of a connection so the receiver can recognise the protocol* : the UTF-8 bytes of the string `Gradle Magic`.[6] The daemon checked the preamble and accepted the connection. The client then sent its first message, a build `Command`, *serialized*. The daemon's message serializer registers dedicated serializers for known message types and falls back to standard Java serialization for any other `Message` (`registry.useJavaSerialization(Message.class)`).[9] Only after the `Command` object had been rebuilt did the connection handler compare `command.getToken()` with the daemon's token and reject a mismatch.[9] That's exactly the pattern in our demo.

The fix, in a commit titled "Check the daemon token before any serialization", moves the token into the handshake. The client now sends the preamble *followed by the 16 token bytes*, and the daemon reads both and compares them as raw bytes before a single object is deserialized. A wrong token now fails with "Peer did not present the expected connection token" and the socket is closed.[6]

TcpIncomingConnector: what the daemon reads first

Before 9.8.1

```
read 12 bytes
if (!equals(PREAMBLE)) fail
// connection accepted
Command c = deserialize(...)
// readObject() code ran
if (!equals(c.getToken(), token))
  reject
```

9.8.1 and 8.14.6

```
read 12 + 16 bytes
if (!startsWith(PREAMBLE)) fail
if (!rest.equals(token))
  fail("Peer did not present
        the expected token")
// only now:
Command c = deserialize(...)
```

Simplified from the diff of commit f28e514 (TcpIncomingConnector.waitForHandshake). The old token comparison lives on in DefaultIncomingConnectionHandler.

The client gets the token from the same daemon registry entry it uses to find the port (`daemon.getToken()`).[6] So a legitimate client on your account, which can read your Gradle user home, works as before. Another local user, who can reach the port but (on a normally configured machine) can't read your Gradle user home, now can't get past the handshake.

Fix 2: worker connections get a token of their own

The worker channel was worse, which is why its advisory says "unauthenticated". Worker processes connected back to the daemon with only the preamble. There was no token at all.[7] The fix, "Check worker token before any serialization similarly to daemon", generates a fresh 16-byte `SecureRandom` token for every worker process the daemon builds, and requires that worker to present it in the handshake.[7]

The interesting design question is how the worker learns its secret without leaking it. The token is added to the worker's configuration object, `WorkerConfig`.[7] Gradle's worker entry point reads that configuration from the worker process's **standard input**.[9] A pipe between parent and child process isn't visible to other users on the machine, unlike a command-line argument, which typically shows up in `ps`.

Who could actually exploit the two deserialization bugs

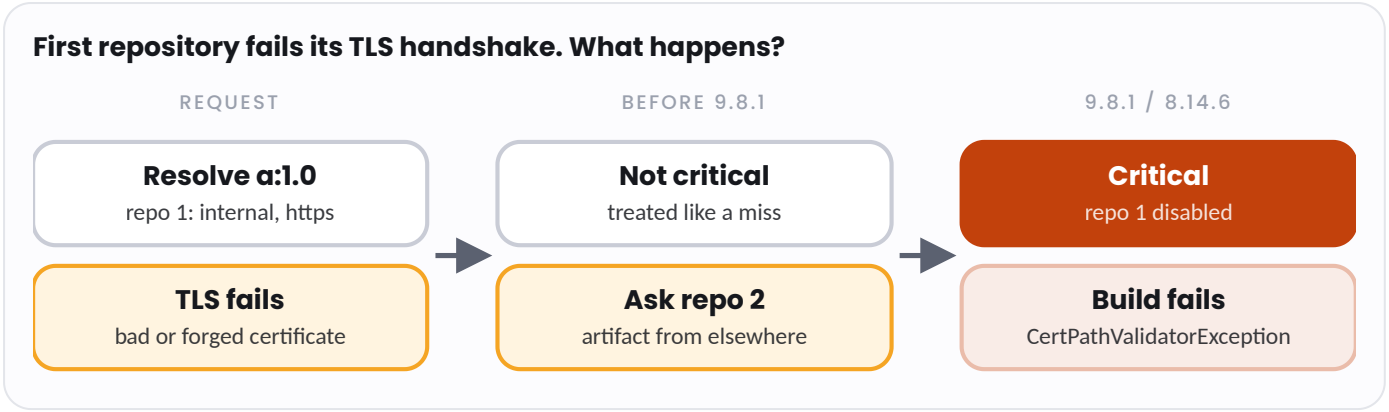
The client-daemon advisory's CVSS vector starts with `AV:L/AC:L/AT:N/PR:L/UI:P`: a **local** attack vector, low complexity, low privileges and passive user interaction. In plain English, the attacker needs an account on the same machine as your daemon. The advisory says so directly: "This vulnerability is not exploitable from a remote network connection." The daemon doesn't need to be building at the time, and code would run with the privileges of the user who owns the daemon. Gradle says shared, multi-user build hosts are most at risk, and the proof-of-concept hasn't been published.[3]

Think of a long-lived CI agent shared by several teams, a university or bootcamp server, or a shared development VM. On those, the build user's daemon is a juicy target: it holds your repository credentials, signing keys from `gradle.properties` and access to whatever the build can publish.

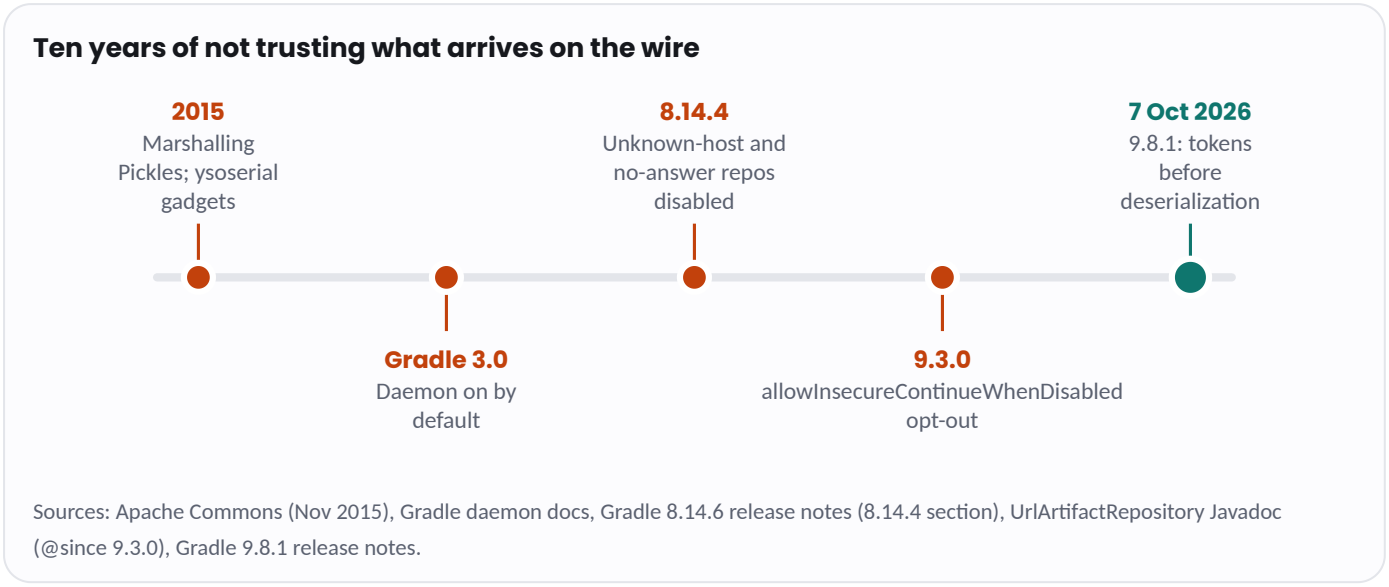
Fix 3: an SSL failure now disables the repository

The repository fix lives in a class called `ResolveExceptionAnalyzer`, which decides whether a failure is "critical" and should disable a repository. Before the fix, only three things counted: a timeout, an HTTP 5xx server error and an unknown host. Everything else, including a **TLS** *≈ Transport Layer Security (often still called SSL): the encryption and certificate checking behind https://* handshake failure, was treated like a normal miss, and the search moved on to the next repository.[8]

The commit title states the new philosophy: "disable them by default on failure unless there's a reason not to". Now *every* failure is critical, except two cases where moving on is legitimate: metadata Gradle couldn't parse, and an HTTP error that isn't a server error or one of the transient client errors (408 and 429).[8] Gradle's new integration tests document the behaviour. A missing module, a 401 Unauthorized response or an unparseable POM in the first repository "does not stop the search". An "untrusted TLS certificate stops the search", and the build fails with a `CertPathValidatorException` instead of quietly downloading the artifact from a plain-HTTP backup repository.[8] A companion commit routes Google Cloud Storage and Amazon S3 repository errors through the same mechanism.[5]



If you really do want a flaky repository to fall through, there's an explicit opt-out: setting `allowInsecureContinueWhenDisabled = true` on that repository lets the search continue. The property has existed since Gradle 9.3.0, and its name tells you what Gradle thinks of it.[12,8] This is also the third round of tightening. Gradle 8.14.4 had already fixed advisories for repositories that failed to answer and repositories with an unknown host.[2]



WHAT IT MEANS FOR YOU

Update the wrapper. For most projects, Gradle's version is pinned in `gradle/wrapper/gradle-wrapper.properties`, so upgrading the Gradle on your laptop isn't enough. The release notes give the command:[1]

```
./gradlew :wrapper --gradle-version=9.8.1 && ./gradlew :wrapper
# still on Gradle 8? use --gradle-version=8.14.6
```

Commit the changed wrapper files, and check that your CI image doesn't install its own Gradle. Teams on 9.0 to 9.7 that can't take the 9.8 feature release have a choice: move to 9.8.1, or buy the security subscription that provides 9.x.2 backports. Teams on 7.x have the same choice with 7.6.7.[3]

Rethink shared build machines. Gradle's own workaround, if you can't upgrade, is to avoid running builds on shared, multi-user machines, and to prefer single-tenant, ephemeral runners *≈ CI machines or containers created fresh for one job and destroyed afterwards, so nothing persists between jobs or users* on CI.[3] That's good advice even after

you upgrade. A token stops other users from talking to your daemon. It doesn't help if the other "user" is a malicious build step running as you.

How exposed is your Gradle setup?

Shared multi-user build host	most at risk (advisory)
Long-lived shared CI agent	other jobs and users can reach loopback
Single-user laptop	attacker would need to be you already
Ephemeral single-tenant runner	advisory's recommended setup

Our reading of the advisory's attack vector (local, low privileges) and its workaround advice. The ordering is illustrative, not from Gradle.

Expect a few builds to fail on purpose. If a build used to "work" because a repository with a broken certificate was silently skipped, 9.8.1 will now stop it. That's the point. Fix the trust store, or remove the repository. Reach for `allowInsecureContinueWhenDisabled` only for a repository that genuinely can't be fixed, and treat it as technical debt.[12]

Check for traces, cautiously. The daemon logs a warning when it fails to process a message from a client socket. Gradle notes that such a log line is not proof of an attack, and that its absence doesn't rule one out.[3]

💥 Watch out for

- **No CVE numbers.** All three advisories were published as GitHub advisories without CVE IDs.[4,3] A scanner or policy that only matches CVE IDs may not flag Gradle 9.8.0 or 8.14.5 as vulnerable. Check the Gradle version yourself.
- `--no-daemon` **isn't a fix.** Even with the daemon disabled, a build can still start a short-lived daemon, and test and isolated-compile workers still connect back over loopback.[10] The worker flaw was in that channel.[1]
- **Your IDE uses the project's Gradle.** The Tooling API runs the version from your wrapper, so an IDE update doesn't patch you. The wrapper bump does.
- **Old daemons keep running.** A daemon started by the old version stays alive until it's idle for three hours. After upgrading on a shared host, run `./gradlew --stop`.[10]
- **Deserialization is a design smell, not a Gradle quirk.** If your own services accept Java-serialized data from the network (old RMI, JMX, caches, custom sockets), the same lesson applies. Oracle's advice is "Do not deserialize untrusted data", and a JDK **serialization filter** *≈ an ObjectInputFilter that tells ObjectInputStream which classes, and how large an object graph, it may accept. Not enabled by default* is the minimum safety net.[13] In our demo, a filter allowing only the expected classes rejected the gadget before its `readObject` ran.

📖 Words from this story

CVSS: the standard 0–10 severity score for security flaws; version 4 is the newest.

Gradle daemon: a long-lived background JVM that runs builds and is reused between them.

Tooling API: the library IDEs use to run Gradle builds and query project models.

Worker processes: extra JVMs the daemon starts (for tests, isolated compilation) that connect back to it.

Loopback: the 127.0.0.1 interface, which only reaches the same machine.

Authentication token: a secret a client must present to prove it may use a service.

Java serialization: the JDK's built-in conversion of object graphs to bytes and back.

Gadget chains: existing classpath classes whose deserialization logic can be chained into an attacker-chosen action.

Dependency confusion: tricking a build into downloading a malicious public package that shares a private package's name.

Preamble: fixed bytes at the start of a connection that identify the protocol.

TLS: the encryption and certificate checking behind https:// (often called SSL).

Ephemeral runners: CI machines or containers created fresh for one job and destroyed afterwards.

Serialization filter: an `ObjectInputFilter` limiting which classes and how much data `ObjectInputStream` accepts.

STORY 2

JEP 544 lands: we read the code that will make Java start warm

JDK 28 · HOTSPOT · PROJECT LEYDEN · INTEGRATED 8 OCT 2026 (FOLLOW-UP TO ISSUE #2) · ~15 MIN READ

Eight days ago, in issue #2, we explained JEP 544 from the outside: the JEP's summary, a few secondary write-ups, and a pull request still under review. On Thursday evening, 8 October, at 21:31 UTC, that pull request became a commit on the JDK's main branch. It changes 179 files and adds 8,641 lines. That means we can stop guessing what "ahead-of-time code compilation" means and read what the JVM will actually do. Some of it confirms what we wrote. One detail corrects it, and several others are things you'll want to know before you build a deployment pipeline around it.

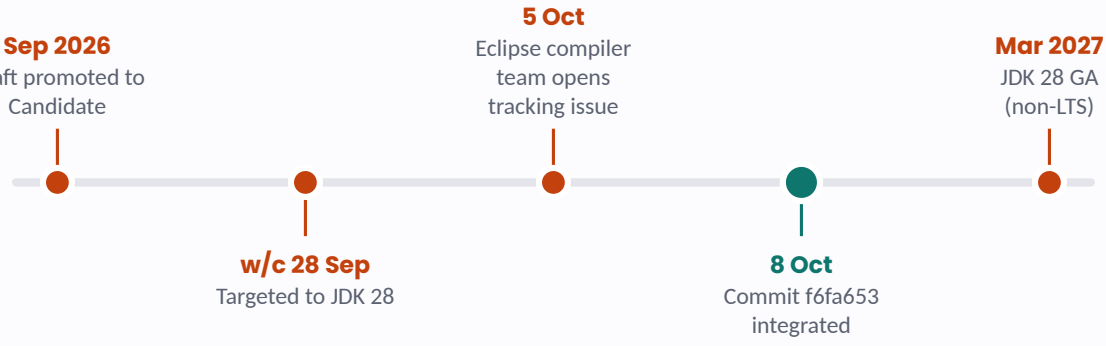
WHAT HAPPENED

Commit `f6fa653`, "8380476: Implement JEP 544: Ahead-of-Time Code Compilation", was integrated into the openjdk/jdk `master` branch on 8 October 2026. Vladimir Kozlov is the author, with ten co-authors including John Rose, Andrew Dinn, Ioi Lam and Aleksey Shipilev, and six reviewers. It touches 179 files, with 8,641 insertions and 1,553 deletions.[16] The heart of it is HotSpot's AOT code cache (about 4,100 changed lines in `aotCodeCache.cpp`) and a new `aotCompileBroker.cpp`, which drives the compilation.[16]

Integrated ≈ *the JEP status that means the implementation has been merged into the JDK source tree for the target release* is the step after *Targeted*. The feature is now part of the code that will become JDK 28, the next feature release, due in March 2027. JDK 28 isn't an LTS release.[17]

You can't download it yet. The latest early-access build tag, `jdk-28+19`, was cut from a commit at 06:26 UTC on 8 October, about fifteen hours before the merge.[18] The first early-access build that should contain JEP 544 is therefore build 20.

JEP 544, from idea to the main branch



Sources: issue #2 (InfoQ roundups), Eclipse JDT issue #5476, openjdk/jdk commit f6fa653, InfoWorld. Build 20 will be the first EA build after the merge.

THE BACKGROUND YOU NEED

Here's the short version of issue #2. A JVM starts by *interpreting* your **bytecode** ≈ *the portable instructions in .class files that the JVM executes*, watches which methods are hot, and compiles them to native code with its two **JIT compilers** ≈ *just-in-time compilers: C1 produces quick, lightly optimised native code; C2 produces heavily optimised code using run-time profiles*. The period before the hot code is fully compiled is **warmup** ≈ *the time after startup during which an application runs slower than its peak because the JIT hasn't yet compiled its hot code*. Every new container starts that work from zero.

Project Leyden attacks it with the **AOT cache** ≈ *a file, created from a training run, that stores work the JVM would otherwise repeat at every startup: loaded and linked classes, heap objects, profiles and now native code*. Since JDK 24 it has stored loaded and linked classes (JEP 483), since JDK 25 method profiles (JEP 515), and JDK 28 adds compiled code with JEP 544. HotSpot labels its compilation levels as **tiers** ≈ *HotSpot's compilation levels: tier 0 is the interpreter, tiers 1–3 are C1 code (tiers 2 and 3 also collect profiles), tier 4 is optimised C2 code*.

The workflow has three phases: a **training run** ≈ *a run of your application on a representative workload whose purpose is to record what the JVM learns*, an **assembly phase** ≈ *the step that turns what the training run recorded into the final AOT cache file*, and then any number of production runs that load the cache. Since JDK 25, `-XX:AOTCacheOutput` does training and assembly in one command.

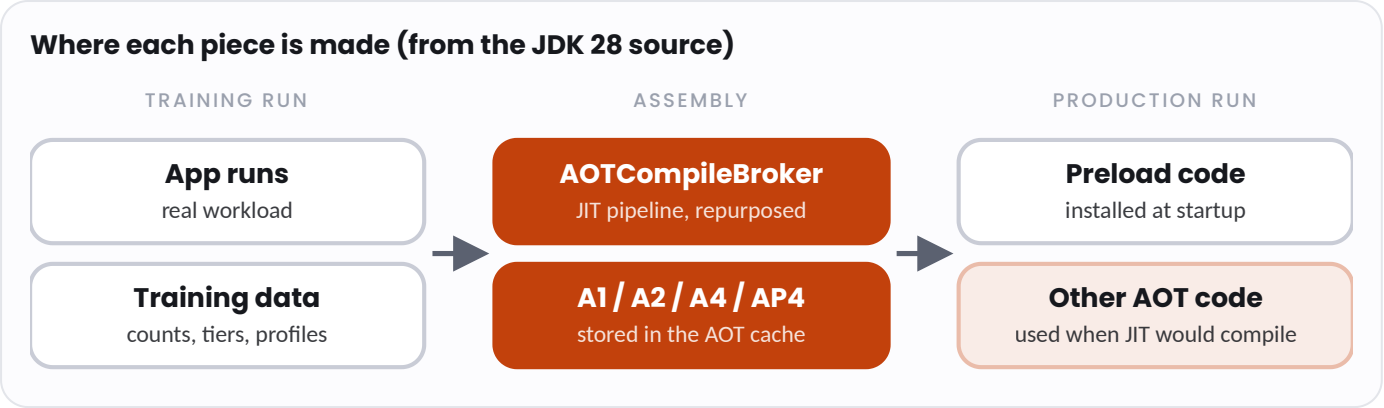
HOW IT WORKS

Correction: the code is compiled during assembly, from the training profile

In issue #2 we wrote, based on secondary summaries, that the cached code "is produced by C2 during the training run". The JEP's own summary, as quoted in the Eclipse compiler team's tracking issue, also speaks of compiling "during a training run".[19] The implementation is more precise, and it matters for how you think about the cache.

During **training**, the JVM only *records* **training data** ≈ *what the training run records about each method: how often it ran, which compilation tiers it reached, and its profile* (the code sets `AOTRecordTraining`). The native code is generated during **assembly**. The comment in `cdsConfig.cpp` reads "Generate AOT code during assembly phase", and only when training data is available.[20] The new `AOTCompileBroker` is the top-level driver. It "runs after metadata and Java object heap have been added to the AOT cache". During that pass, normal JIT compilation is suppressed and "the JIT pipeline is repurposed to create AOT code".[21]

Which methods get compiled? The broker walks the training data and selects each method that reached a level during training. For final C1 and C2 code, it compiles a method "only when there was relevant compilation during training". It skips native and abstract methods, and methods whose class isn't linked. It sorts them by the order in which they were first compiled during training, "presuming the methods that compiled earlier are more important".[21] The new code has its own tier names: A1, A2 and A4 for AOT versions of tiers 1, 2 and 4, plus **preload** code (AP4), which is C2 code meant to be installed right at startup.[21,22]



With the one-step `-XX:AOTCacheOutput` command, training and assembly happen back to back, so the difference is invisible on the command line. It still matters, because it means **the code reflects the profile at the moment training ended**. Anything your training workload didn't exercise isn't in the cache.

In production: start with AOT code, end with JIT code

When a production JVM opens a cache that contains code, it switches on `AOTCompileEagerly`, which the flag description says compiles "methods as soon as possible during production run". It also starts a separate loading thread.[23,22] Preload code is processed asynchronously by default. A diagnostic flag, `AOTPreloadBlocking`, makes startup wait until "all preloaded code is done loading".[22]

The most interesting part is a comment in C2's parser, `parse1.cpp`. It explains why AOT code isn't the end state. (An **nmethod** *≈ HotSpot's internal name for one compiled method: a block of native code plus the metadata the JVM needs to run and discard it* is HotSpot's name for a compiled method.)[24]

> "Even the highest tier of AOT code (A4) incorporates an invocation counter on entry to an AOT nmethod. This is necessary because AOT optimizations are performed on less accurate information than JIT optimizations."

The comment gives two examples. The AOT compiler "refuses to fold" many `static final` values as constants, because it can't predict them ahead of time. And it only sees the profile up to the point the cache was built, while production "may execute much longer". So each A4 method counts its own calls. When the count passes a limit, the JIT recompiles the method with the live profile and replaces the AOT version, "so as to reoptimize with the fullest possible information" and reach "the same peak performance as pure JIT-based workloads".[24]

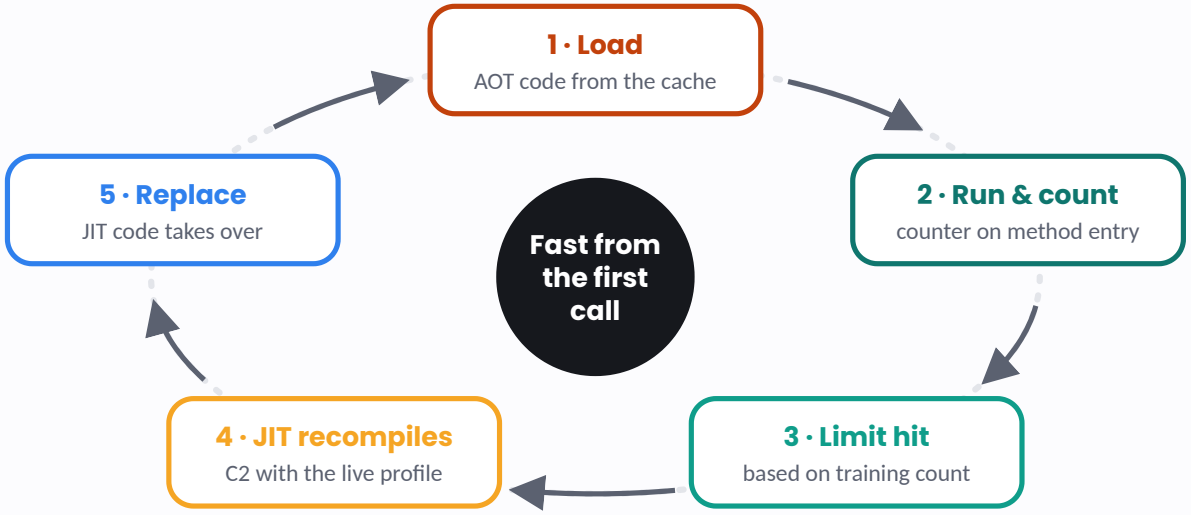
The limit depends on how often the method ran during training. The code uses a saturating curve: `100 + n / (1 + n / 1,000,000)`, where *n* is the training invocation count and both constants are tunable diagnostic flags.[24,22] We worked a few values through the formula with the default settings:

Calls during training (n)	AOT calls before JIT recompiles
1,000	≈ 1,100
100,000	≈ 91,000
1,000,000	≈ 500,100

Calls during training (n)	AOT calls before JIT recompiles
100,000,000	≈ 990,200

So a method that was barely used during training gets replaced quickly, and even the hottest method is swapped for fresh JIT code after about a million calls. The counter only sits on the root method of each compiled unit. Inlined calls aren't counted, which is why the authors expect "counting overhead" to be small.[24]

The life of one AOT-compiled method in production



Where it switches itself off

The biggest practical lesson from the source is how many conditions can silently disable the cached code. Native code bakes in assumptions about the machine and the JVM's configuration. The cache records them, and the code's own comment says "we take a pessemistic [sic] stand and drop the whole cache if any of these are changed".[23]

If production differs in…	What happens
CPU architecture	Only x64, AArch64 and RISC-V 64 are supported at all
CPU features (e.g. vector instructions)	"AOT Code Cache disabled: cpu features are incompatible"
Garbage collector	Disabled: "it was created with GC = …"
Assertions (`-ea`), page size, many intrinsic flags	Disabled with a config-mismatch message
JVMTI ≈ *the JVM Tool Interface: the native API debuggers, profilers and Java agents use to inspect and modify a running JVM* capabilities (breakpoints, redefinition)	AOT method loading disabled: "incompatible JVMTI state"

If production differs in…	What happens
`--enable-preview`	"AOT Code Caching is not supported with Valhalla"
Class loaded by a custom class loader	That method is skipped at assembly time

Our summary of checks in aotCodeCache.cpp/.hpp, cdsConfig.cpp and arguments.hpp on master, 13 Oct 2026.[23,25,26]

"Intrinsic flags" control **intrinsics** ≈ *hand-written machine-code versions of specific JDK methods (crypto, CRC32, array copies) that the JIT uses instead of compiling the Java code*, which the AOT code may have used. A few others deserve a sentence too. The platform list is a correction to issue #2, which said "AArch64 and x64 only, at first": the integrated code also enables RISC-V 64.[23] For **compressed oops** ≈ *the JVM's trick of storing 64-bit object references as 32-bit offsets from a heap base, which saves memory*, a new flag, `AOTCompatibleOopCompression`, is "set by default when assembling an AOT cache which contains code". It forces a mode in which AOT code works whatever heap address range production gets.[22] The JVMTI check compares capabilities such as breakpoints, local-variable access and class redefinition or retransformation, which debuggers and many Java agents enable.[26] And the preview check is blunt. In JDK 28, `--enable-preview` is how you opt into Valhalla's value classes, and the code treats any preview-enabled run as "Valhalla enabled" and turns AOT code caching off.[25,23]

The good news is that none of these fail silently if you ask. Failure messages go to the `aot` log, so `-Xlog:aot` shows you why code wasn't used.[23]

The switches

The user-facing commands are unchanged from JDK 25. The new knobs are all **diagnostic flags** ≈ *HotSpot options meant for troubleshooting, which only work after -XX:+UnlockDiagnosticVMOptions and may change without notice*. The main one, `AOTCodeCaching`, is enabled automatically when an AOT cache is created or used with training data. You can turn it off to compare.[22,23]

```
# JDK 28 EA (build 20+), not yet runnable: commands from the source
java -XX:AOTCacheOutput=app.aot -jar app.jar     # train + assemble
java -XX:AOTCache=app.aot -Xlog:aot -jar app.jar  # use, log why not

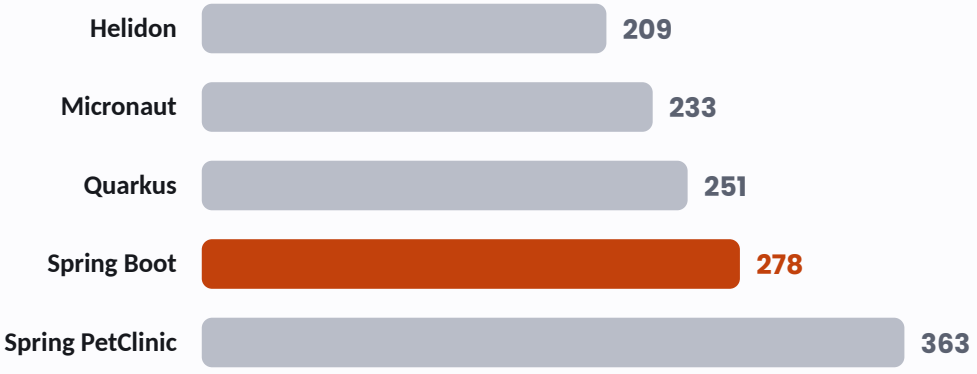
# A/B test: same cache, without the compiled code
java -XX:AOTCache=app.aot -XX:+UnlockDiagnosticVMOptions \
     -XX:-AOTCodeCaching -jar app.jar
```

We couldn't run these. JDK 28 builds can't be downloaded from our sandbox, and no build with JEP 544 exists yet.

Early numbers, from a contributor

The pull request's review thread includes measurements by Dingli Zhang, one of the commit's co-authors, on two RISC-V machines with two cores each. Startup time was normalised so that a plain run is 1,000 (lower is better). With the AOT cache including code, the five workloads came in between 209 and 363 on the SG2044 machine. On both machines, the cache with code beat the cache without code on all five workloads.[27]

Startup with AOT cache + code, RISC-V SG2044, 2 cores (plain run = 1,000; lower is better)



Contributor measurements posted on openjdk/jdk PR #30778 (Dingli Zhang), not an independent benchmark. 278 means the Spring Boot app started in 27.8% of its normal time.

Other reviewers reported no peak-performance regressions on PetClinic, with lower startup CPU and faster warmup, and a Quarkus user saw peak response times reached with roughly half the requests in one example.[27] All of these are pre-release measurements from people working on or with the feature.

WHAT IT MEANS FOR YOU

Your training run is now your compiler's input. Because AOT code is generated from training data, a training run that only boots the app (Spring's `-Dspring.context.exit=onRefresh`) gives you fast class loading but little compiled request-handling code. Issue #2 already made this point. The source makes it concrete: methods that never reached a compilation tier during training are simply not compiled.[21]

Train like you deploy. Use the same JDK build, GC, CPU type, heap settings, `-ea` setting and Java agents in training as in production. If your production pods run an observability agent and your training run doesn't, check `-Xlog:aot` for a JVMTI mismatch before you trust your benchmarks.[23,26]

Spring Boot's "extract the jar" step matters more. Methods of classes loaded by a custom class loader are skipped.[23] An executable Spring Boot fat jar loads your classes through Boot's own launcher class loader, not the JDK's application class loader. That fits with Spring Boot's AOT cache documentation, which already tells you to run the *extracted* form of the application, as we showed in issue #2.

Peak performance is still the JIT's job. The counters mean a long-running service ends up on JIT-compiled code anyway. JEP 544 is about the first minutes, not about replacing C2.[24]

💥 Watch out for

- **Not in a build yet.** Build 19 predates the merge, so wait for JDK 28 EA build 20.[18] JDK 28 itself ships in March 2027 and isn't an LTS.[17]
- **Diagnostic flags can change.** `AOTCodeCaching`, `AOTPreloadBlocking`, `AOTCodeInvokeBase` and the rest are diagnostic options, so treat them as tuning knobs for experiments, not stable configuration.[22]
- **Preview features turn it off.** If you experiment with value classes on JDK 28, you don't get AOT code in the same run.[25,23]
- **"Pessimistic" means all or nothing.** One configuration mismatch drops all the cached code, not just some methods.[23]
- **The JEP text may differ.** We read the implementation, not the final JEP page, which our sandbox couldn't open. Where the two disagree, the code is what will run.

WHAT HAPPENS NEXT

Build 20 should give everyone a chance to try it. Expect the `java` launcher documentation to catch up: the integration commit doesn't change the `java` man page.[16] Tools are moving too. The Eclipse compiler team is tracking JEP 544 for its Java 28 milestone.[19] And the next Leyden step is already visible in the code's own to-do comments, such as giving A4 methods with long-running loops a **backedge counter** *≈ a counter of how many times a loop jumps back to its start, which the interpreter uses to spot hot loops that deserve compiling* like the interpreter's.[24]

📖 Words from this story

Integrated: the JEP status meaning the implementation is merged into the JDK source for the target release.

JIT compilers: C1 (quick, light optimisation) and C2 (heavy, profile-guided optimisation), which compile hot code while the program runs.

Warmup: the slow period after startup before hot code is fully compiled.

AOT cache: the file from a training run holding classes, heap objects, profiles and now native code.

Tiers: HotSpot's compilation levels; tier 0 interpreter, tiers 1–3 C1, tier 4 C2.

Training run: a run on a representative workload whose purpose is to record what the JVM learns.

Assembly phase: the step that turns training output into the final AOT cache.

Training data: per-method records of invocation counts, tiers reached and profiles.

JVMTI: the JVM Tool Interface used by debuggers, profilers and Java agents.

Compressed oops: 64-bit references stored as 32-bit offsets from a heap base.

Diagnostic flags: HotSpot troubleshooting options that need `-XX:+UnlockDiagnosticVMOptions` and may change.

Bytecode: the portable instructions in `.class` files that the JVM executes.

nmethod: HotSpot's internal name for one compiled method and its metadata.

Intrinsics: hand-written machine code for specific JDK methods that the JIT uses instead of compiling their Java code.

Backedge counter: counts loop iterations so the JVM can spot hot loops.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Structured Concurrency's review closed:** JEP 543's Proposed-to-Target review for finalising `StructuredTaskScope` in JDK 28 ended 13 October, and the "Targeted" announcement is the next thing to watch (issue #5 has the explainer).[28]
- **ZGC's adaptive heap JEPs close review today:** JEPs 545 and 546, from issue #8, end their Proposed-to-Target review on 14 October.[28]
- **"Structured Concurrency Looms" at Devoxx Belgium:** Alan Bateman and Viktor Klang's talk covers lessons from the previews and a new lower-level API that may underpin a future structured concurrency API.[29]
- **Ron Pressler's Devoxx keynote, "Java – The Year of the Runtime!":** argues that as AI agents make syntax less central, runtime performance and operability matter more, with updates on Valhalla, Leyden and Lilliput.[30]
- **WildFly 42 Beta1:** adds virtual-thread support to its Jakarta Concurrency 3.1 implementation.[31]
- **Jackson CVE-2026-68497:** a JSON string bound to `Duration` or `XMLGregorianCalendar` can burn seconds to minutes of CPU per request with a default `ObjectMapper`; fixed in 2.18.10, 2.21.6, 2.22.2, 3.1.6 and 3.2.2.[32]
- **Infinispan 16.3.0 and Micronaut 5.2.2:** a new release of the distributed cache and a Micronaut framework update based on Micronaut Core 5.2.15.[28]

---

1. **Gradle 9.8.1 Release Notes**, Gradle; source at tag `v9.8.1` (tag commit 7 Oct 2026). Advisory links: **GHSA-mvvg-497x-hmj8**, **GHSA-j5m7-59rp-24f5** (titles as listed in the notes; these two advisory pages didn't load from our sandbox).
2. **Gradle 8.14.6 release notes**, Gradle, at tag `v8.14.6` (includes the 8.14.4 advisories section).
3. **Deserialization of untrusted Java objects before authentication in the client - daemon communication (GHSA-xwqc-3h47-hg64)**, Gradle, published 7 Oct 2026.
4. **Submit: Gradle 9.8.1 Patches Three High-Rated Vulnerabilities...**, The Machine Herald (editorial review notes citing the three advisories), Oct 2026.
5. Gradle git tags read 14 Oct 2026: `v9.8.1` (7 Oct), `v8.14.6` (8 Oct), `v9.8.0` (23 Sep); commit **74a7720** "Wrap GCP & S3 resolution errors into HTTP to reuse repository disabling mechanism".
6. **Commit f28e514: "Check the daemon token before any serialization"**, Vlad Chesnokov, Gradle (TcpIncomingConnector, TcpOutgoingConnector, DefaultDaemonConnector).
7. **Commit 651110a: "Check worker token before any serialization similarly to daemon"**, Vlad Chesnokov, Gradle (WorkerConfig, DefaultWorkerProcessBuilder).
8. **Commit 8a6b9d9: "Harden repository disabling..."**, Vlad Chesnokov, Gradle (ResolveExceptionAnalyzer and RepositoryFailureScopeIntegrationTest).
9. Gradle source at tag `v9.8.1`: **Daemon.java**, **DefaultIncomingConnectionHandler.java**, **DaemonMessageSerializer.java**, **GradleWorkerMain.java**.
10. **Gradle Daemon**, Gradle User Manual (source `gradle_daemon.adoc` at `v9.8.1`).
11. **Repository disabling**, Gradle User Manual (source `graph_resolution.adoc` at `v9.8.1`).
12. **UrlArtifactRepository.getAllowInsecureContinueWhenDisabled()**, Gradle API source, `@since 9.3.0`.
13. **Addressing Deserialization Vulnerabilities**, Java SE 21 Core Libraries guide, Oracle. Our demo of readObject ordering and a filter was compiled and run on OpenJDK 21.0.12.1.
14. **Apache Commons statement to widespread Java object de-serialisation vulnerability**, Bernd Eckenfels and Gary Gregory, The Apache Software Foundation Blog, 10 Nov 2015.
15. **frohoff/ysoserial README**, Chris Frohoff et al., GitHub.
16. **Commit f6fa653: "8380476: Implement JEP 544: Ahead-of-Time Code Compilation"**, Vladimir Kozlov et al., openjdk/jdk, 8 Oct 2026 (read via git, including `git show --stat`).

17. **Java 28 starts to take shape**, Paul Krill, InfoWorld, 24 Sep 2026.
18. openjdk/jdk tag `jdk-28+19`, commit dated 8 Oct 2026 06:26 UTC; latest `jdk-28+*` tag as of 14 Oct 2026 (git ls-remote).
19. **[28] JEP 544: Ahead-of-Time Code Compilation**, eclipse-jdt/eclipse.jdt.core issue #5476, 5 Oct 2026 (quotes the JEP summary).
20. **src/hotspot/share/cds/cdsConfig.cpp**, openjdk/jdk `master` (as changed by f6fa653).
21. **src/hotspot/share/compiler/aotCompileBroker.cpp**, openjdk/jdk `master`, 13 Oct 2026.
22. Flag definitions on openjdk/jdk `master`, 13 Oct 2026: **cds_globals.hpp**, **compiler_globals.hpp**, **c2_globals.hpp**, **globals.hpp** (`AOTCompatibleOopCompression`).
23. **src/hotspot/share/code/aotCodeCache.cpp** and **aotCodeCache.hpp**, openjdk/jdk `master`, 13 Oct 2026.
24. **src/hotspot/share/opto/parse1.cpp** (`count_aot_code_calls` and its comment), openjdk/jdk `master`; worked examples of `scale_limit` are our arithmetic.
25. **src/hotspot/share/runtime/arguments.hpp** (`is_valhalla_enabled` returns `enable_preview()`), openjdk/jdk `master`.
26. **src/hotspot/share/prims/jvmtiManageCapabilities.cpp**, openjdk/jdk `master`.
27. **8380476: Implement JEP 544: Ahead-of-Time Code Compilation, PR #30778**, openjdk/jdk (review thread measurements by Dingli Zhang, dchuyko and Delawen).
28. **Java News Roundup: Grails 8, Open Liberty, GlassFish, Infinispan, LangChain4j, Spock**, Michael Redlich, InfoQ, 12 Oct 2026.
29. **Structured Concurrency Looms**, Alan Bateman and Viktor Klang, Inside.java (Devoxx Belgium 2026), 12 Oct 2026.
30. **Java - The Year of the Runtime!**, Ron Pressler, Inside.java (Devoxx Belgium 2026), 11 Oct 2026.
31. **New WildFly 42 Beta release**, Brian Stansberry, wildfly.org, 8 Oct 2026.
32. **CVE-2026-68497: Denial of Service in jackson-databind**, HeroDevs vulnerability directory, published 11 Sep 2026.